Task 1

It shows how each move stores a tuple which represents changes in health to the player and the enemy as a game. The turns also swap depending on whose turn it is using random import to decide the enemy's move.

```
Enemy selected:  normal
Player health: 50
Enemy health: 40

PLAYER TURN
Select move...
normal
You selected:  normal
Player health: 50
Enemy health: 20

ENEMY TURN
Enemy selected:  last stand
Player health: 20
Enemy health: 5

PLAYER TURN
Select move...
normal
You selected:  normal
Player health: 20
Enemy health: -15

GAME OVER: Player wins.
```

```
PLAYER TURN
Select move...
last stand
You selected:  last stand
Player health: 25
Enemy health: 20

ENEMY TURN
Enemy selected:  normal
Player health: 5
Enemy health: 20

PLAYER TURN
Select move...
special
You selected:  special
Player health: 10
Enemy health: 10

ENEMY TURN
Enemy selected:  special
Player health: 0
Enemy health: 15

GAME OVER: Enemy wins.
```

```
ENEMY TURN
Enemy selected:  heal
Player health: 45
Enemy health: 60

PLAYER TURN
Select move...
last stand
You selected:  last stand
Player health: 30
Enemy health: 30

ENEMY TURN
Enemy selected:  normal
Player health: 10
Enemy health: 30

PLAYER TURN
Select move...
last stand
You selected:  last stand
Player health: -5
Enemy health: 0

DRAW.
```

Task 2

```python
# Generate a sorted list of integers for testing
sorted_list = list(range(1000000))

# Test cases
target = 10000  # Random target value
linear_time = timeit.timeit(lambda: linear_search(sorted_list, target), number=100
binary_time = timeit.timeit(lambda: binary_search(sorted_list, target), number=100
```

algorithms_project_file

```
C:\Users\musta\AppData\Local\Programs\Python\Python314\python.exe C:\Users\musta\PycharmP
99999
-1
1
8
Linear search execution time: 0.018653 seconds
Binary search execution time: 0.000173 seconds
```

Target = 10 000,
range = 1000 000

```python
# Generate a sorted list of integers for testing
sorted_list = list(range(100))

# Test cases
target = 25  # Random target value
linear_time = timeit.timeit(lambda: linear_search(so
binary_time = timeit.timeit(lambda: binary_search(so
```

algorithms_project_file

```
C:\Users\musta\AppData\Local\Programs\Python\Python314\pyth
99999
-1
1
8
Linear search execution time: 0.000064 seconds
Binary search execution time: 0.000063 seconds
```

Target = 25, range = 100

Linear always takes more time however, the less the range, the closer the time gets. Binary search is more efficient due to it being more complex and it requires the data to be sorted. If the binary search is run on unsorted data, it can perform just as poorly as linear search and it may produce error.

Task 3

Results:

```
Sorted array: [11, 12, 22, 25, 64]
Sorted array: [1, 11, 12, 22, 64]
Bubble Sort Execution Time: 2.887424 seconds
Insertion Sort Execution Time: 1.209180 seconds
```

```
Sorted array: [1, 4, 5, 7, 11, 12, 22, 25, 64, 100, 101, 102, 103, 104, 105, 106, 109]
Sorted array: [1, 4, 5, 7, 10, 11, 12, 22, 64, 100, 101, 102, 103, 104, 105, 106, 109]
Bubble Sort Execution Time: 2.929592 seconds
Insertion Sort Execution Time: 1.247982 seconds
```

The insertion sort is performing better than the bubble sort as shown in the results using 2 different lists. The insertion sorting is better due to it building the final sorted array one item at a time. While the bubble sorting is comparing adjacent elements and not the whole list together. That ends up running slower than insertion due to how slow it is to compare two at a time compared to the whole list. For example, consider the list [2, 3, 4, 5, 6, 7, 8, 1]. Insertion sorting will take the 1 and places it at the right spot while bubble will check each adjacent elements until it reaches the 1 and swaps it with the 8 and so on until it places the 1 at the start.

Task 4

```
Data Size = 100
Insertion sort + Linear search execution time: 0.019430 seconds
bubble sort + Linear search execution time: 0.019236 seconds
Insertion sort + binary search execution time: 0.000797 seconds
bubble sort + binary search execution time: 0.000447 seconds
Data Size = 200
Insertion sort + Linear search execution time: 0.017385 seconds
bubble sort + Linear search execution time: 0.016569 seconds
Insertion sort + binary search execution time: 0.000629 seconds
bubble sort + binary search execution time: 0.000393 seconds
Data Size = 400
Insertion sort + Linear search execution time: 0.008620 seconds
bubble sort + Linear search execution time: 0.008728 seconds
Insertion sort + binary search execution time: 0.000687 seconds
bubble sort + binary search execution time: 0.000410 seconds
Data Size = 8000
Insertion sort + Linear search execution time: 0.001219 seconds
bubble sort + Linear search execution time: 0.000816 seconds
Insertion sort + binary search execution time: 0.000876 seconds
bubble sort + binary search execution time: 0.000559 seconds
```

As comparing the different sizes, we can see that binary search is always faster than linear search and the bigger the size, the closer the time difference is. That aligns with Task 2. It seems that bubble sort and binary search is the fastest. The reason could be the difference between data sizes can drastically affect the sorting algorithm.